

Heaven's light is our guide"

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

Compiler Design

Course No. : 701

Chapter 2: A Simple One-Pass Compiler

Prepared By : Julia Rahman

2.1 Overview

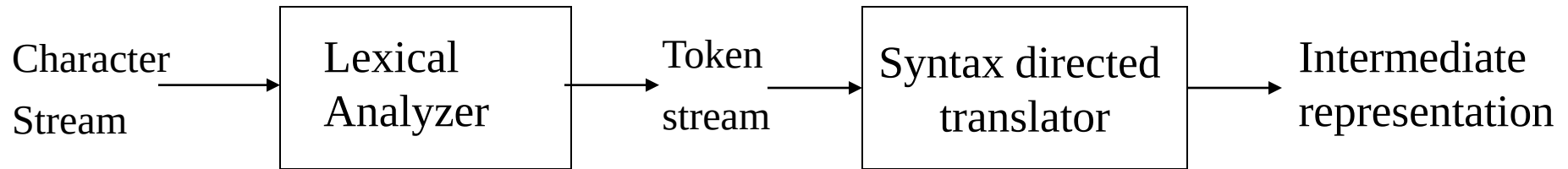


Figure 2.1: Structure of a compiler front end

2.2 Syntax Definition

+ Context-free grammar:

- ✓ Utilized to describe the Syntactic Structure of a Language
- ✓ Describes hierarchical structure with production rules.
- ✓ Has four components:
 1. A Set of **Tokens** or **Terminal** Symbols
 2. A Set of **Non-terminals**
 3. A Set of **Production Rules**
Each Rule Has the Form
 $NT \rightarrow \{T, NT\}^*$
 4. A Non-terminal Designated As the **Start Symbol**

+ **Example 2.1:** “Lists the digits separated by plus or minus symbol”. The following grammar describes the syntax of the expression. The productions are:

list $\rightarrow$ *list* + *digit*

list $\rightarrow$ *list* - *digit*

list $\rightarrow$ *digit*

digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

So, we can write

list $\rightarrow$ *list* + *digit* | *list* - *digit* | *digit*

2.2 Syntax Definition

- ✓ A string of tokens is a sequence of zero or more tokens.
- ✓ The string containing with zero tokens, written as ϵ , is called **empty** string.
- ✓ A grammar derives **strings** by beginning with the start symbol and repeatedly replacing the non terminal by the right side of a production for that non terminal.
- ✓ The token strings that can be derived from the start symbol form the **language** defined by the grammar.
- ✓ Given a CF grammar we can determine the set of all strings (sequences of tokens) generated by the grammar using derivation
 - We begin with the start symbol
 - In each step, we replace one nonterminal in the current sentential form with one of the right-hand sides of a production for that nonterminal

2.2 Syntax Definition

- ❖ Using the CFG defined on the earlier slide, we can derive the string: $9 - 5 + 2$ as follows:

$list \rightarrow list + digit$	P1 : $list \rightarrow list + digit$
$\rightarrow list - digit + digit$	P2 : $list \rightarrow list - digit$
$\rightarrow digit - digit + digit$	P3 : $list \rightarrow digit$
$\rightarrow 9 - digit + digit$	P4 : $digit \rightarrow 9$
$\rightarrow 9 - 5 + digit$	P4 : $digit \rightarrow 5$
$\rightarrow 9 - 5 + 2$	P4 : $digit \rightarrow 2$

- This is an example ***leftmost derivation***, because we replaced the leftmost nonterminal (underlined) in each step.
Likewise, a ***rightmost derivation*** replaces the rightmost nonterminal in each step.

2.2 Syntax Definition

- ❖ This derivation could also be represented via a Parse Tree (parents on left, children on right)

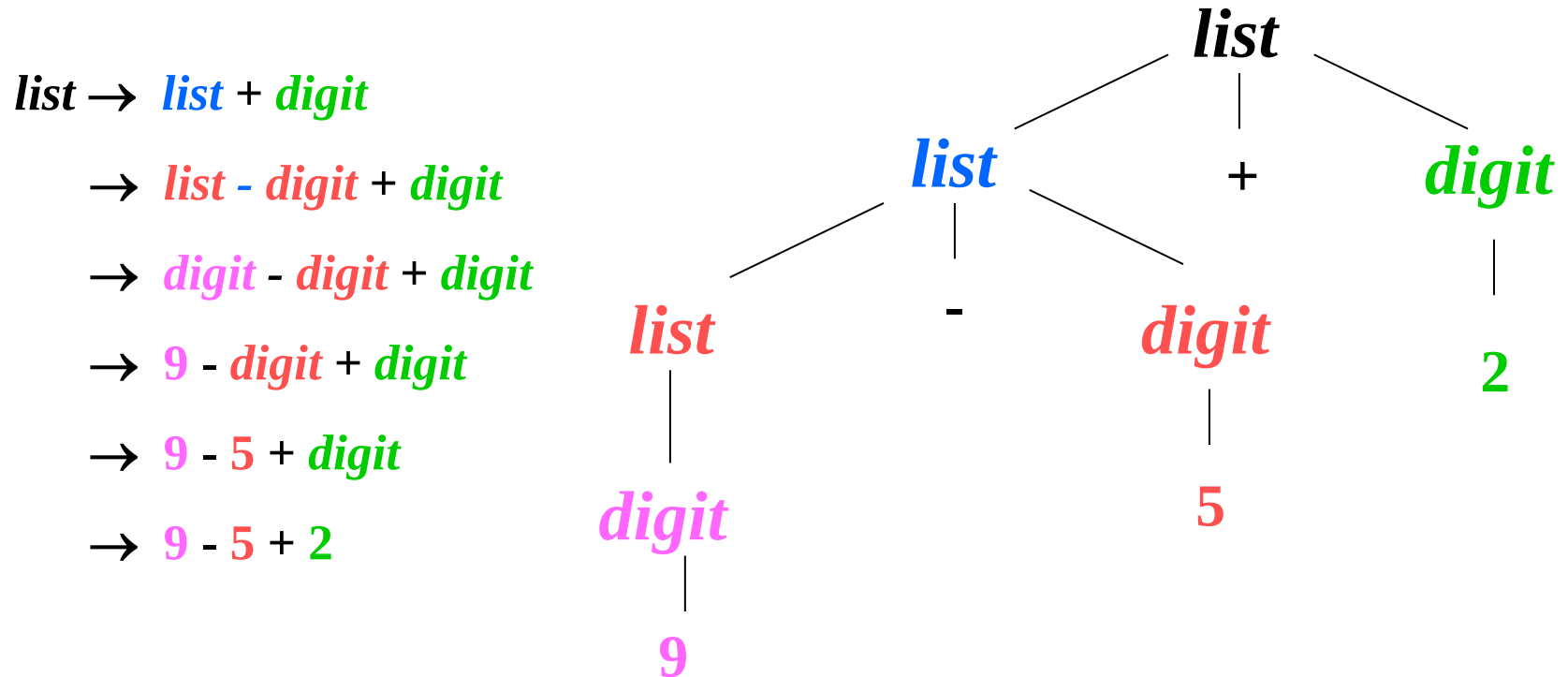
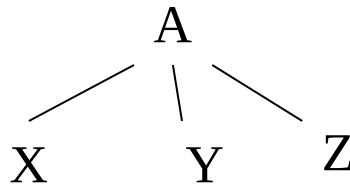


Figure 2.2: Parse tree for 9-5+2 according to the grammar

2.2 Syntax Definition

Parse Tree:

- ✓ A parse tree pictorially shows how the start symbol of a grammar derives a string in the language.
- ✓ If nonterminal A has a production $A \rightarrow XY Z$, then a parse tree may have an interior node labeled A with three children labeled X , Y , and Z , from left to right:



- ✓ More Formally, a Parse Tree for a CFG Has the Following Properties:
 - Root Is Labeled With the Start Symbol
 - Leaf Node Is a Token or ϵ
 - Interior Node is a Non-Terminal
 - If $A \rightarrow X_1 X_2 \dots X_n$ is a production, then node A has immediate *children* $X_1, X_2, \dots, X_n$ where X_i is a (non)terminal or ϵ (ϵ denotes the *empty string*)

2.2 Syntax Definition

Ambiguity:

- ✓ A grammar may have more than one parse tree generating a given string of tokens.

- ✓ **Grammar:**

$string \rightarrow string + string \mid string - string \mid 0 \mid 1 \mid \dots \mid 9$

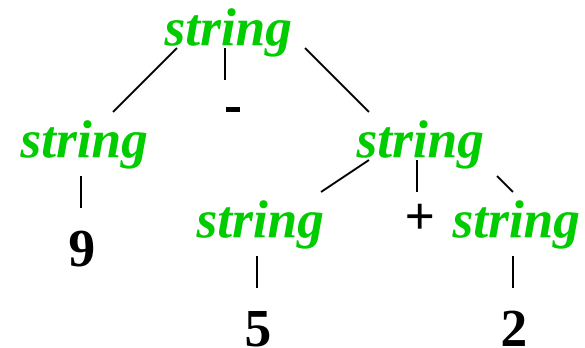
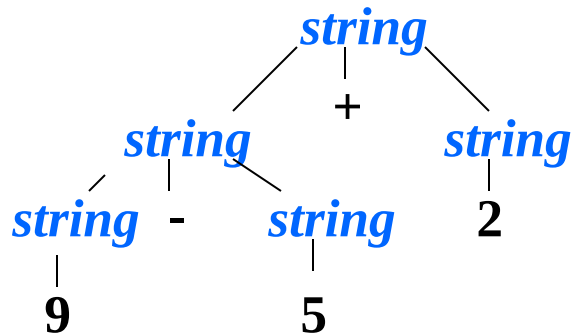


Figure 2.3: Two parse trees for 9-5+2

2.2 Syntax Definition

Associativity of Operators:

- ✓ Left-associative operators have left-recursive productions

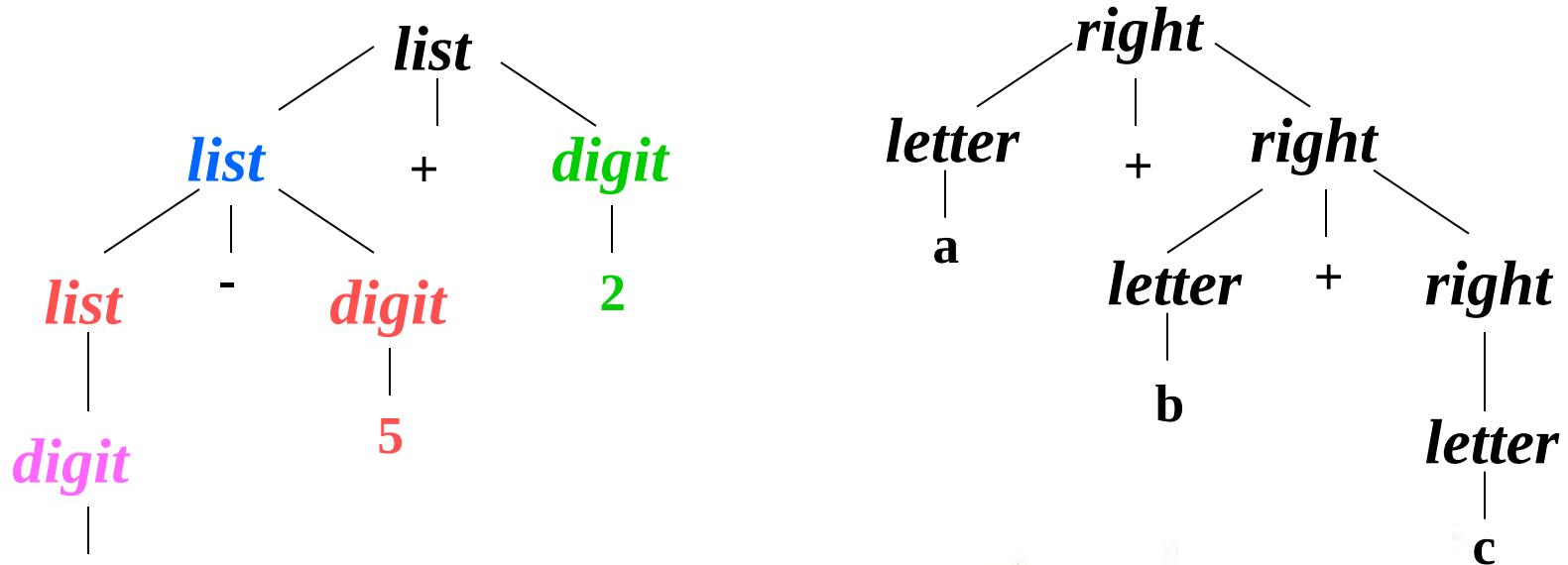
$left \rightarrow left + term \mid term$

String $a+b+c$ has the same meaning as $(a+b)+c$

- ✓ Right-associative operators have right-recursive productions

$right \rightarrow term = right \mid term$

String $a+b+c$ has the same meaning as $a+(b+c)$



9 Figure 2.4: Parse trees for left- and right-associative grammars

2.2 Syntax Definition

Precedence of Operators:

Typically $\left\{ \begin{array}{l} () \\ * / \\ + - \end{array} \right.$ is precedence order

- ✓ What does $9 + 5 * 2$ mean?

This can be incorporated into a grammar via rules:

expr $\rightarrow$ *expr* + *term* | *expr* – *term* | *term*
term $\rightarrow$ *term* * *factor* | *term* / *factor* | *factor*
factor $\rightarrow$ *digit* | (*expr*)
digit $\rightarrow$ 0 | 1 | 2 | 3 | ... | 9

- ✓ **Precedence Achieved by:**

expr & term for each precedence level

- ✓ Rules for each are left recursive or associate to the left

2.2 Syntax Definition

stmt → id := *expr*
| if *expr* then *stmt*
| if *expr* then *stmt* else *stmt*
| while *expr* do *stmt*
| begin *opt_stmts* end

2.3 Syntax-Directed Translation

- ✓ An Syntax Directed Translation is a context free grammar + attributes + semantic rules
 - Attributes associated to symbols
 - Rules associated to productions
- ✓ Uses a CF grammar to specify the syntactic structure of the language
- ✓ An attribute is said to be ...
 - **Synthesized:**
 - Depends on the children of node and itself
 - Can be evaluated in any bottom-up order (e.g. depth first)
 - Also terminals may have synthesized attributes.
 - **inherited** if its value at a parse-tree node is determined by the parents and siblings of the node and itself. Terminals may not have inherited attributes.

Postfix notation:

The postfix notation for an expression E can be defined inductively as follows:

1. If E is a variable or constant then **Postfix(E) = E**
2. If E is E_1 **op** E_2 then **Postfix(E)**
= **Postfix(E_1 op E_2) = Postfix(E_1) Postfix(E_2) op**
3. If E is (E_1) then **Postfix(E) = Postfix(E_1)**

2.3 Syntax-Directed Translation

Syntax-Directed Definition:

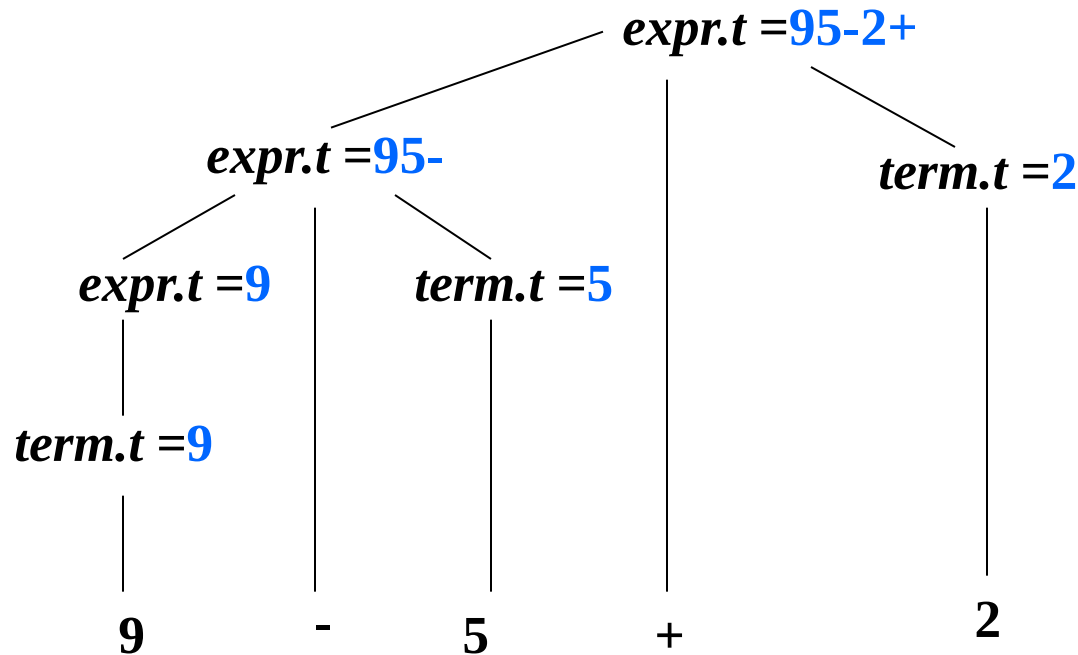
- ✓ formal specification of translation
 - specify the translation of a construct in terms of attributes associated with syntactic components
- ✓ Each Production has a Set of Semantic Rules
- ✓ Each Grammar Symbol has a set of Attributes
- Each Production Rule of the CFG Has a Semantic Rule

Production	Semantic Rule
$expr \rightarrow expr + term$	$expr.t := expr.t \parallel term.t \parallel '+'$
$expr \rightarrow expr - term$	$expr.t := expr.t \parallel term.t \parallel '-'$
$expr \rightarrow term$	$expr.t := term.t$
$term \rightarrow 0$	$term.t := '0'$
$term \rightarrow 1$	$term.t := '1'$
....	
$term \rightarrow 9$	$term.t := '9'$

Note: Semantic Rules for $expr$ define t as a “synthesized attribute” i.e., the various copies of t obtain their values from “children t ’s”

2.3 Syntax-Directed Translation

✚ Semantic Rules are Embedded in Parse Tree (Annotated Parse Tree) :



❖ *expr.t* denotes attribute *t* of node *expr* in the parse tree

- ✓ It starts at the root and recursively visits the children of each node in left-to-right order
- ✓ The semantic rules at a given node are evaluated once all descendants of that node have been visited.
- ✓ A parse tree showing all the attribute values at each node is called ***annotated parse tree***.

2.3 Syntax-Directed Translation

Example:

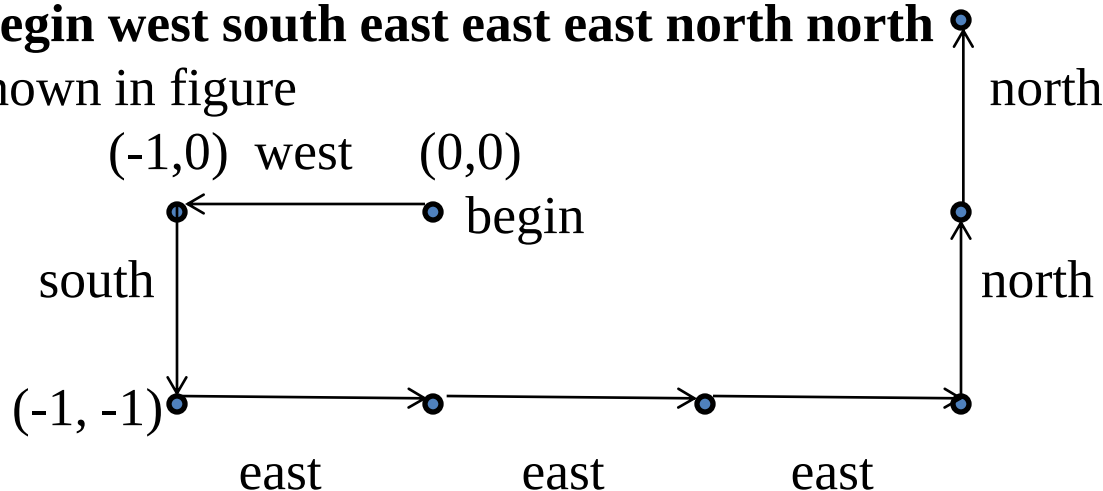
Suppose a robot can be instructed to move one step east, north, west or south from its current position. A sequence of such instructions is generated by the following grammar:

$$\text{seq} \rightarrow \text{seq instr} \mid \mathbf{begin}$$
$$\text{instr} \rightarrow \mathbf{east} \mid \mathbf{north} \mid \mathbf{west} \mid \mathbf{south}$$

Changes in the position of the robot on input

Begin west south east east east north north

Are shown in figure



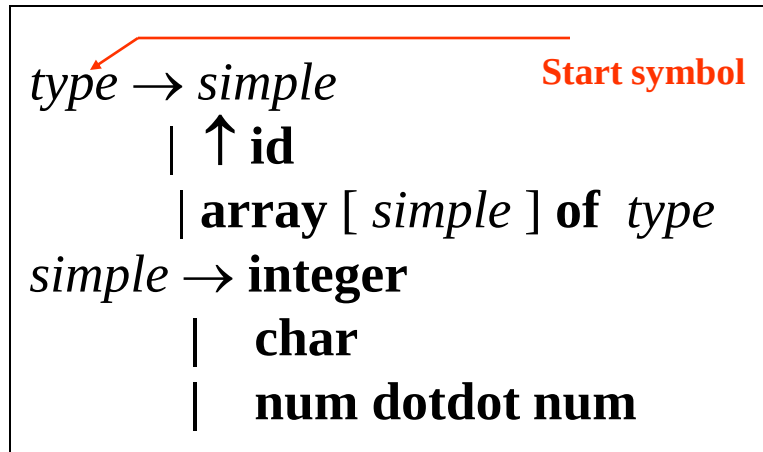
2.4 Parsing

- ✓ Parsing is the process of determining if a string of tokens can be generated by a grammar.
- ✓ Parser must be capable of constructing the tree.
- ✓ For any CF grammar there is a parser that takes at most $O(n^3)$ time to parse a string of n tokens
- ✓ Two types of parser:
 - 1) **Top-down:**
 - Starts at root
 - Proceeds towards leaves
 - Easily constructed by hands
 - 2) **Bottom-up:**
 - Starts at leaves
 - Proceeds towards root
 - Efficient parsers are mostly bottom-up
 - Automatic parser generation tools tend to produce bottom-up parsers
- ❖ **Lookahead symbol** – Current token of input is being scanned

2.4 Parsing

✚ **Top-Down Parsing** $\Rightarrow$ Parse tree / derivation of a token string occurs in a top down fashion.

✓ For Example, Consider:



✓ Suppose **input** is :

array [num dotdot num] of integer

✓ The top-down construction of a parse tree is done by starting with the root, labeled with the starting nonterminal, and repeatedly performing the following two steps:

- 1) At node N, labeled with nonterminal A, select one of the productions for A and construct children at N for the symbols in the production body.
- 2) Find the next node at which a sub tree is to be constructed, typically the leftmost unexpanded nonterminal of the tree.

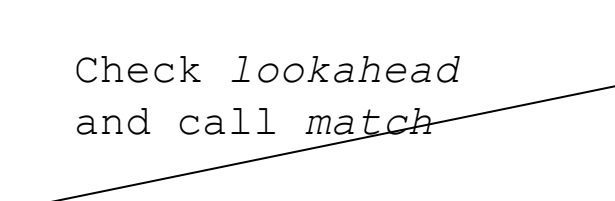
2.4 Parsing

Example Top Down Parser (Execution Step 1)

*Check lookahead
and call `match`*

`match('array')`

`type()`

A line connects the `match()` function in the `type()` call to the `match('array')` call, indicating that the lookahead token is being used to match the expected type.

Input: **array** [num dotdot num] of integer

 ↑

lookahead

The input string is shown with the word 'array' highlighted. An arrow points from the word 'array' to the word 'lookahead' below it, indicating that 'array' is the current lookahead token.

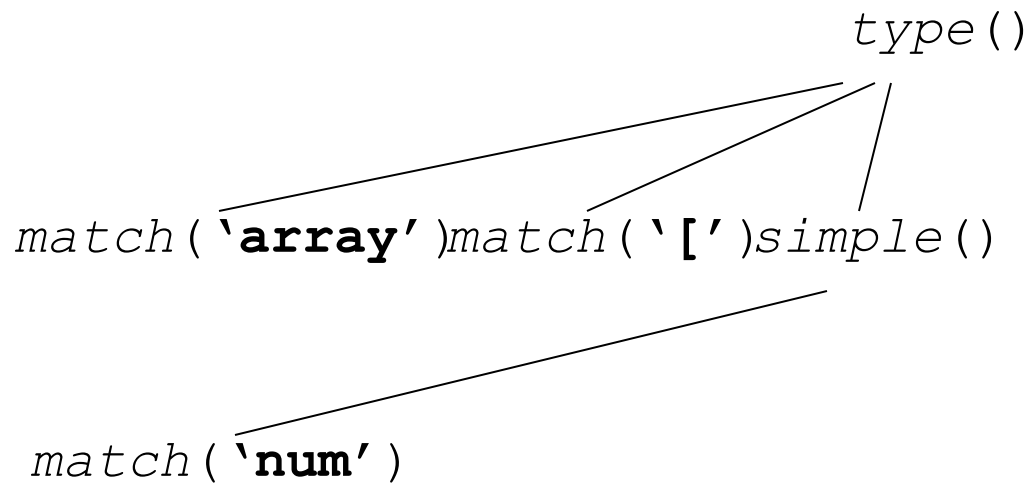
(Execution Step 2)

Diagram illustrating the relationship between `match()` and `type()` functions. The `match()` function is shown with two lines pointing to the `array` and `[` elements in the expression `match('array')match('[')`.

Input: **array** $\left[\begin{matrix} \text{num} & \text{dotdot} & \text{num} \\ \uparrow & & \end{matrix} \right]$ of integer
 lookahead

2.4 Parsing

(Execution Step 3)

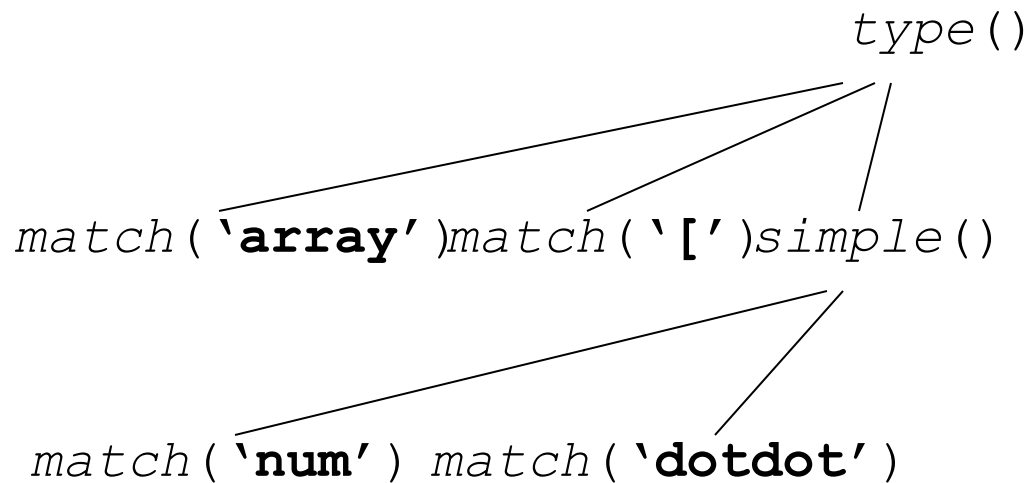


Input: **array** **[** **num** **dotdot** **num** **]** **of** **integer**

↑
lookahead

2.4 Parsing

(Execution Step 4)

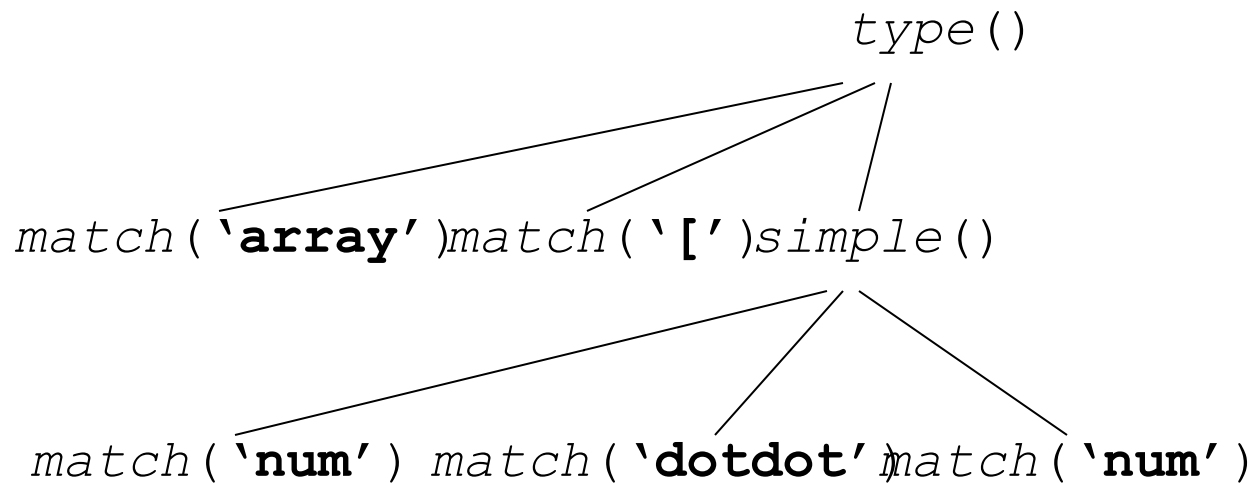


Input: **array** **[** **num** **dotdot** **num** **]** **of** **integer**

↑
lookahead

2.4 Parsing

(Execution Step 5)

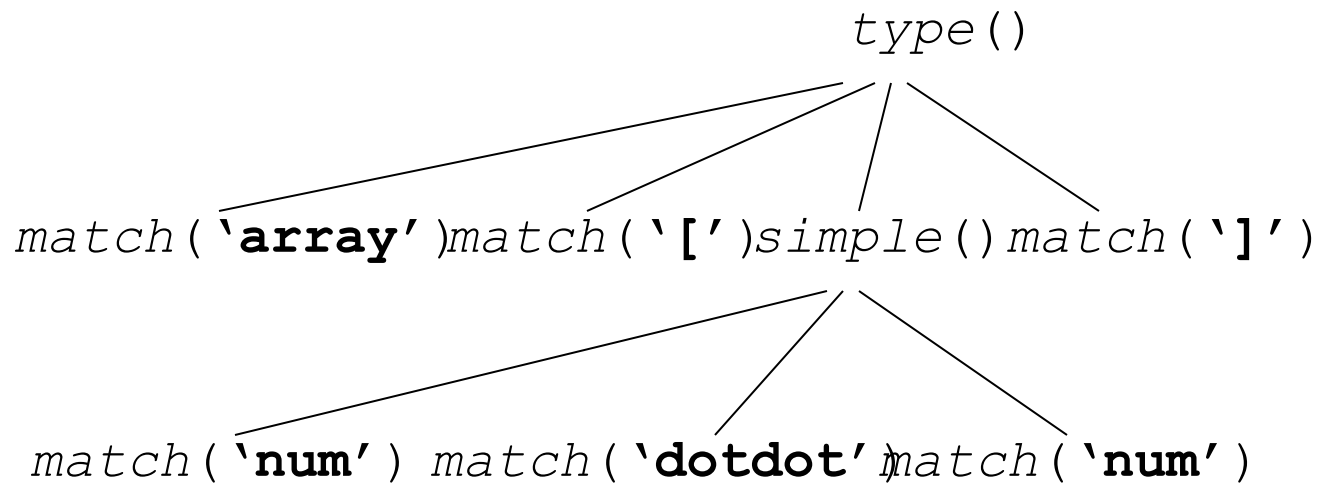


Input: **array** **[** **num** **dotdot** **num** **]** **of** **integer**

↑
lookahead

2.4 Parsing

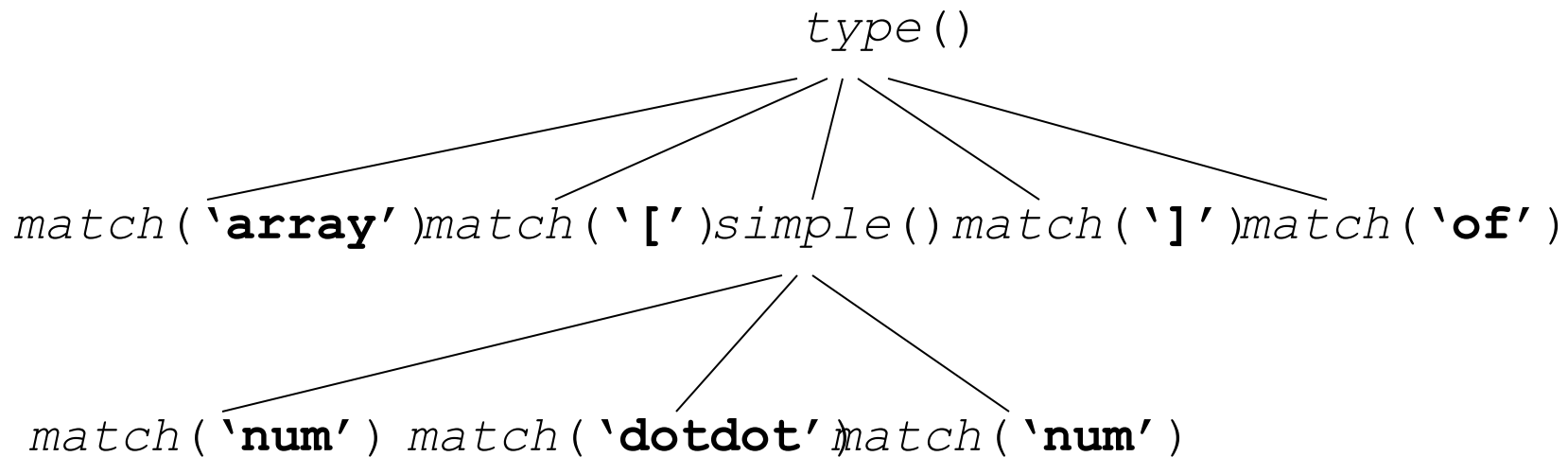
(Execution Step 6)



Input: **array** **[** **num** **dotdot** **num** **]** **of** **integer**

↑
lookahead

(Execution Step 7)

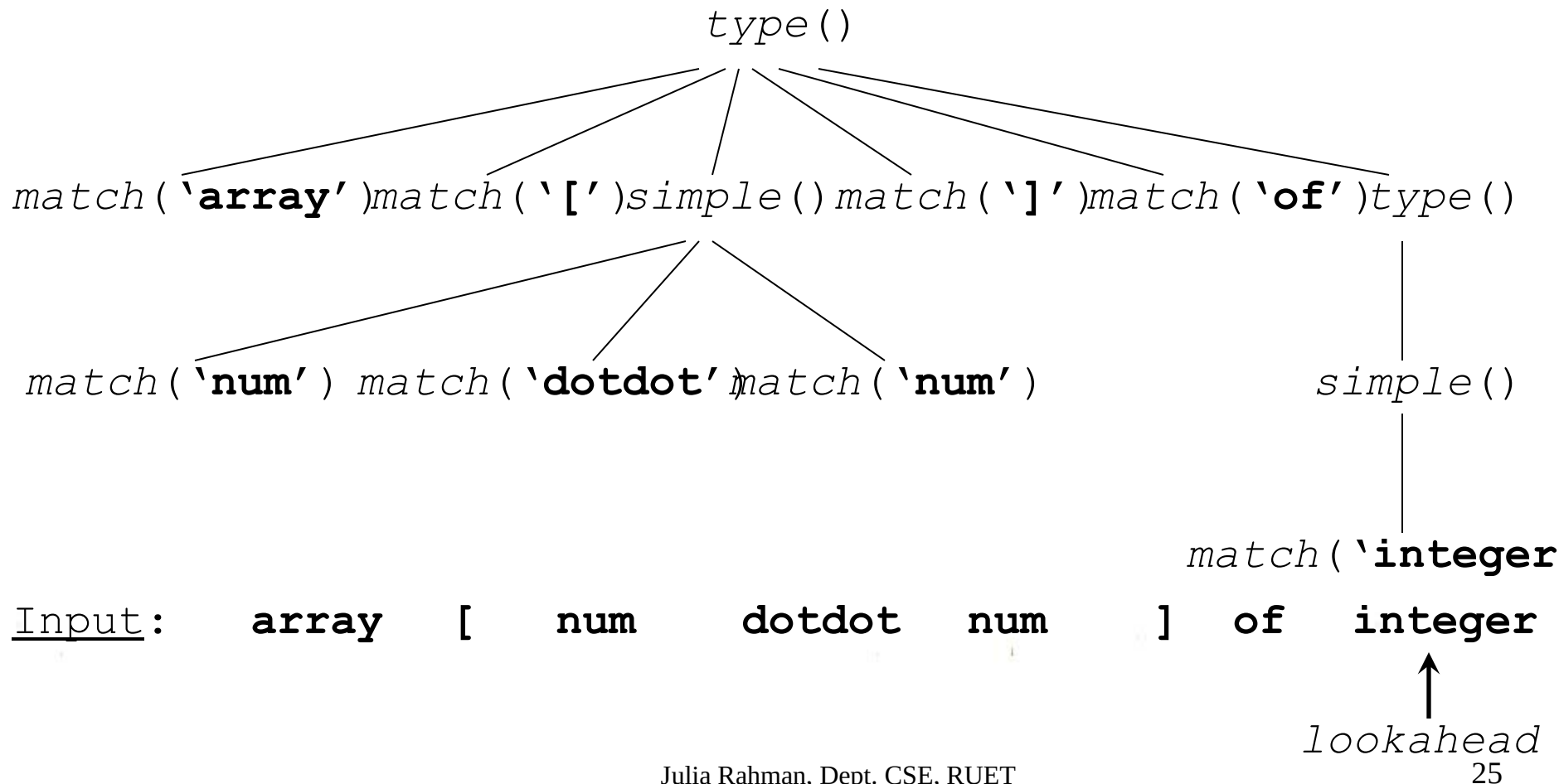


Input: **array** [**num** **dotdot** **num**] **of** **integer**

$\uparrow$
lookahead

2.4 Parsing

(Execution Step 8)



2.4 Parsing

Predictive Parsing:

- ✓ ***Recursive descent parsing*** is a top-down parsing method
 - Execute a set of recursive procedures to process the inputs.
 - Every nonterminal has one (recursive) procedure responsible for parsing the nonterminal's syntactic category of input tokens
 - When a nonterminal has multiple productions, each production is implemented in a branch of a selection statement based on input look-ahead information
- ✓ ***Predictive parsing*** is a special form of recursive descent parsing where we use one lookahead symbol to unambiguously determine the parse operations
- ✓ Example Predictive Parser (Grammar)

```
type → simple
      | ↑ id
      | array [ simple ] of type
simple → integer
      | char
      | num dotdot num
```

2.4 Parsing

Example Predictive Parser (Program Code)

```
procedure match(t : token);  
begin  
  if lookahead = t then  
    lookahead := nexttoken()  
  else error()  
end;
```

```
procedure type();  
begin  
  if lookahead in { 'integer', 'char', 'num' } then  
    simple()  
  else if lookahead = '^' then  
    match('^'); match(id)  
  else if lookahead = 'array' then  
    match('array'); match('['); simple();  
    match(']'); match('of'); type()  
  else error()  
end;
```

```
procedure simple();  
begin  
  if lookahead = 'integer' then  
    match('integer')  
  else if lookahead = 'char' then  
    match('char')  
  else if lookahead = 'num' then  
    match('num');  
    match('dotdot');  
    match('num')  
  else error()  
end;
```

2.4 Parsing



Left Recursion:

- ✓ When a production for nonterminal A starts with a self reference then a predictive parser loops forever

$$\begin{array}{l} A \rightarrow A \alpha \\ \quad | \beta \\ \quad | \gamma \end{array}$$

- ✓ We can eliminate left recursive productions by systematically rewriting the grammar using right recursive productions

$$\begin{array}{l} A \rightarrow \beta R \\ \quad | \gamma R \\ R \rightarrow \alpha R \\ \quad | \varepsilon \end{array}$$